

# Compiler-1 보고서

## I. 파일 설명

#1 <prac.l> : 입력 문자열을 정의된 토큰으로 리턴해주는 Lex 분석기를 정의한 파일이다. lex 명령어를 통해 'lex.yy.c' 파일을 생성한다.

#2 <prac.y>: C언어의 일부 문법(syntax)을 정의해 놓은 파일로, yacc 명령어에 -d 옵션을 추가해 'y.tab.c' 뿐만 아니라 'y.tab.h'를 생성할 수 있다. 각각 syntax 분석을 하는 파일과 토큰을 정의 해놓은 헤더 파일이다.

'prac.l'에는 정의된 토큰을 확인할 수 있도록 'y.tab.h'를 include 하였고, 입력 소스의 마지막에 들어오는 EOF를 원만하게 처리하기 위해 yywrap 함수를 추가하여 1을 리턴하도록 하였다. 'y.tab.c'에는 error를 처리해 줄 yyerror 함수를 추가해 error message를 출력하게 하였다. 또한, 'pracmain.c' 파일을 만들어 yyparse 함수를 실행할 수 있게 하였다.

## II. Test Case

1.

```
~$ ./a.out  
int a=3;
```

설명: 문법 규칙에 정의 되어 있는 declaration으로, 'declaration\_specifiers(int) init\_declarator\_list\_opt (declarator('a') ASSIGN('=') initializer(3)) SEMICOLON' 으로 이루어져 있다. 따라서 정상적으로 종료된다.

2.

```
;  
syntax error
```

설명: 입력은 function\_definition 또는 declaration이 한 번 이상 나와야 한다. 따라서 SEMICOLON만 나온 경우 start symbol인 program이 성립되지 않아 "syntax error"가 발생한다.

3.

```
int float;
```

설명: 정의 되어 있는 declaration으로, type\_specifier(int) declaration(float)이 합해져 declaration\_specifiers가 되고 init\_declarator\_list('') SEMICOLON(';')가 순서대로 나오는 구성을 가진다. 이는 규칙에 맞기 때문에 정상적으로 종료된다.

4.

```
auto int static float;
```

설명: declaration\_specifiers는 type\_specifier와 storage\_class\_specifier가 문법에 맞는 여러 번 발생할 수 있고, 따라서 auto(storage\_class\_specifier), int(type\_specifier), static(storage\_class\_specifier), float(type\_specifier)가 연속적으로 나올 수 있으며 정상 종료된다.

5.

```
struct a;
```

설명: declaration에서 declaration\_specifiers에 STRUCT\_SYM, init\_declarator\_list\_opt에 'a', 마지막으로 SEMICOLON이 구조에 맞게 쓰여 정상 종료된다.

6.

```
struct a  
syntax error
```

설명: declaration에서 마지막에 SEMICOLON이 존재하지 않아 "syntax error" 메시지가 출력된다.

7.

```
auto struct a;
```

설명: declaration으로, storage\_class\_specifier(auto) type\_specifier(struct)가 합쳐져 declaration\_specifiers가 되고, init\_declarator\_list\_opt에는 'a', 마지막으로 SEMICOLON으로 정상 종료된다.

8.

```
union a { int a;};
```

설명: declaration으로, type\_specifier에는 union, IDENTIFIER에는 'a', 그리고 LR RR 사이 struct\_declarator\_list에 type\_specifier(int), struct\_declarator\_list(a), SEMICOLON, 마지막에 존재하는 SEMICOLON 순으로 적절하게 배치되어 정상 종료된다.

9.

```
union {int a, float b};  
syntax error
```

설명: declaration으로, struct\_declarator\_list에 struct\_declarator가 COMMA를 사용해 여러 번 선언이 가능하다. 하지만, struct\_declaration 마지막에 SEMICOLON이 존재하지 않아 "syntax error" 메시지가 출력된다.

10.

```
typedef int a;
typedef a b;
syntax error
```

설명: typedef int a; 는 declaration의 구조로 정상적으로 동작한다. 하지만 typedef를 사용해도 a가 정의되어 있는 타입으로 처리되지 않아 'typedef a b;'에서 "syntax error"가 발생한다.

11.

```
void hamsoo(float a, int b){ }
```

설명: function\_definition 구조로, 'declaration\_specifiers(void) declarator(direct\_declarator(hamsoo) LP parameter\_type\_list\_opt(float a, int b) RP)' 구조로 정의 되어 있기 때문에 정상 종료된다.

12.

```
void hamsoo(float, int){}
```

설명: function\_definition 구조로, parameter\_list COMMA parameter\_declaration에서 (declaration\_specifiers(float) abstract\_declarator\_opt(''))가 parameter\_list가 되고, (declaration\_specifiers(int) abstract\_declarator\_opt(''))가 parameter\_declaration가 된다. 이 두 개가 정의된 규칙에 맞게 COMMA로 연결되어 정상 종료된다.

13.

```
h(int a=3){}
syntax error
```

설명: function\_definition 구조로, 'LP declarator RP'에서의 declarator에는 type\_specifier가 포함되는 문법 규칙이 존재하지 않아 "syntax error"가 발생한다.

14.

```
void hamsoo(float a, int b){
    int c;
    c=2;
}
```

설명: function\_definition으로, 'declaration\_specifiers(void) declarator(direct\_declarator(hamsoo) LP parameter\_type\_list\_opt(float a, int b) RP) compound\_statement({ int c; c=2; })' 구성이다. 문법에 정의 되어 있어 정상 종료된다.

15.

```
int hamsoo(int a){  
    for(i=0;i<=5;i++){  
}
```

설명: function\_definition으로 declaration\_specifiers declarator compound\_statement 구조를 가지고, compound\_statement 안에 iteration\_statement가 존재한다. iteration\_statement 중 FOR\_SYM(for) LP for\_expression(expression\_opt SEMICOLON expression\_opt SEMICOLON expression\_opt) RP statement({ })로 정의된 규칙에 맞는다. 또한, 각각의 expression\_opt는 assignment\_expression, relational\_expression, postfix\_expression에 맞게 쓰여져 있어 정상 종료된다.

16.

```
h(){ for(;;){} }
```

설명: function\_declaration 안에 iteration\_statement가 포함된 구조이다. for문의 for\_expression은 'expression\_opt SEMICOLON expression\_opt SEMICOLON expression\_opt'인데, 문법에서 expression\_opt은 공백이 가능하기 때문에 정상적인 종료가 된다.

17.

```
ham(){ int a, float b;}  
syntax error
```

설명: compound\_statement의 declaration\_list\_opt 안의 declaration\_list에서는 COMMA가 아닌 SEMICOLON을 통해 각 declaration이 구분되어 연속적으로 나올 수 있다. declaration\_list에는 COMMA와 관련된 규칙이 존재하지 않기 때문에 "syntax error" 메시지가 발생한다.

18.

```
h(){ for(;;) }  
syntax error
```

설명: function\_definition 안의 iteration\_statement 중 for문은 'FOR\_SYM LP for\_expression RP statement' 구조로, 위의 상황에서는 statement를 포함하고 있지 않아 "syntax error" 메시지가 출력된다.

19.

```
int hamsoo(int a)
{ for(int i=0;i<=5;--i){}
syntax error
```

설명: function\_definition으로 declaration\_specifiers declarator compound\_statement 구조를 가지고, compound\_statement 안에 iteration\_statement가 존재한다. for\_expression에 존재하는 첫 번째 구간 assignment\_expression에서 type\_specifier가 정의되어 있지 않아 “syntax error” 메시지가 발생한다.

20.

```
for(i=1;i<=5;i++){ }
syntax error
```

설명: iteration\_statement가 function\_definition 내부가 아닌 단독으로 사용되는 것은 정의되어 있지 않아 “syntax error”가 발생한다.

21.

```
int hamsoo(){
    if(a==3){ }
    else{}
}
```

설명: function\_definition 안에 selection\_statement가 존재하는 구조이다. IF\_SYM LP expression RP statement ELSE\_SYM statement가 적절하게 적혀 있으며 if에서의 expression도 equality\_expression에 따라 적절하게 쓰여져 있어 정상 종료된다.

22.

```
int main(){
    switch(a){
        case 1: break;
        case 2: continue;
        default: a=0;
    }
}
```

설명: function\_definition 안에 selection\_statement가 존재하는 구조이다. SWITCH\_SYM LP expression RP statement가 잘 구성되어 있으며, expression은 primary\_expression의 IDENTIFIER이다. 또한, statement에서의 compound\_statement 안에 다시 labeled\_statement가 3번 반복된다. 이때 나타나는 jump\_statement와 assignment\_expression 역시도 잘 쓰여 있어 정상 종료된다.

23.

```
int main(){
    switch(a){
        case 1:break;
        case 2: continue;
        default: a=0, b=0;
    }
}
```

syntax error

설명: function\_definition 안에 selection\_statement가 존재하는 구조이다. expression\_statement에는 COMMA를 통해 여러 수식이 나올 수 있는 규칙이 정의되지 않아 “syntax error” 메시지가 출력된다.

24.

```
main(){
    while(a<=5);
}
```

설명: declaration\_specifiers가 없는 declarator와 compound\_statement 구조이다. compound\_statement 안에는 iteration\_statement가 존재하고 정의되어 있는 구조이다. 소괄호 안에 있는 수식 역시도 정의를 제대로 따르고 있어 정상 종료된다.

25.

```
main(){ while(a<=5+3*2); }
```

설명: function\_definition에서의 declaration\_specifiers가 없는 declarator와 compound\_statement 구조이다. compound\_statement 안에는 iteration\_statement가 존재하고 정의되어 있는 구조이다. ‘a’와 ‘5+3\*2’가 적절한 연산 규칙에 따라 적용되고 계산되어 정상 종료된다.

26.

```
main(){ while(a<=2){ default: break; return; ;;;;;;;;;;}}
```

설명: function\_definition 구조로 compound\_statement에 iteration\_statement가 들어있다. WHILE\_SYM 다음에 나오는 소괄호 속의 수식이 적절하고, while문 compound\_statement의 statement\_list에서 statement가 반복해서 나올 수 있어 labeled\_statement와 jump\_statement, 연속적인 expression\_statement가 나오는 것이 가능하다. 따라서 정상 출력된다.

27.

```
m(){ do case 3:break; while (1+2+3+4+5);}
```

설명: iteration\_statement의 DO\_SYM statement WHILE\_SYM LP expression RP SEMICOLON 구조에서 statement에 ‘case 3: break;’, expression에 ‘1+2+3+4+5’이 적절한 위치와 구조로 적혀 있어 정상 종료된다.

28.

```
int a=++3;
```

설명: declaration에서 unary\_expression(PLUSPLUS unary\_expression)이 적절하게 적혀 있어 정상 종료된다.

29.

```
int a=(3 && 5);
```

설명: declaration의 init\_declarator\_list\_opt에서 'LR initializer\_list RR'을 통해 logical\_and\_expression이 적절한 위치에 들어가게 되어 정상 출력된다.

30.

```
int a=(3 & 5);  
syntax error
```

설명: declaration의 init\_declarator\_list\_opt에서 아직 bitwise\_and\_expression에 '&' 연산에 대한 정의가 되어 있지 않아 "syntax error" 메시지가 출력된다.

31.

```
int a=(3<<5);  
syntax error
```

설명: declaration의 init\_declarator\_list\_opt에서 아직 shift\_expression에 '<<' 연산에 대한 정의가 되어 있지 않아 "syntax error" 메시지가 출력된다.

32.

```
int *****a[234]();
```

설명: declaration 구조로 direct\_declarator 앞에 여러 개의 pointer가 존재할 수 있으며, 뒤에는 대괄호, 소괄호가 쌍을 이뤄 연속적으로 나올 수 있다. 또한 대괄호에는 constant\_expression\_opt이 나올 수 있고, 위는 INTEGER\_CONSTANT로 정상적인 종료가 된다.

33.

```
int a = c->b;
```

설명: declaration 구조로, initializer에 postfix\_expression ARROW IDENTIFIER 구조로 되어 있다. 따라서 정상적인 종료가 된다.

34.

```
int b = sizeof(iinntt);
```

설명: declaration으로, initializer가 unary\_expression의 SIZEOF\_SYM unary\_expression 구조를 가진다. '(iinntt)'는 primary\_expression의 'LP expression RP' 구조로 인식이 되어 정상 출력된다.

35.

```
struct a {int a=3;};  
syntax error
```

설명: declaration으로, type\_specifier에 STRUCT\_SYM, IDENTIFIER에 'a', LR과 RR 사이의 struct\_declaration\_list가 존재하는 구조이다. struct\_declaration\_list의 struct\_declaration에서는 type\_specifier(int) struct\_declarator\_list(a=3) SEMICOLON의 구성을 갖는데, 이때, struct\_declarator\_list에는 declarator만 가능하고 '=(ASSIGN)'과 관련된 문법이 정의되어 있지 않아 "syntax error" 메시지가 출력된다.

36.

```
enum { a, b=4, c=b+c++, a };
```

설명: declaration으로, type\_specifier에 'ENUM\_SYM LR enumerator\_list RR' 구조를 갖는다. enumerator\_list는 enumerator가 반복해서 나올 수 있고, enumerator는 'IDENTIFIER ASSIGN expression' 규칙과 'IDENTIFIER' 단독으로도 사용할 수 있는 규칙이 존재한다. 따라서, 정의된 문법에 적합하고 'b+c++' expression 역시도 정의된 규칙을 따르기 때문에 정상 종료된다.